00
PROJET 01
M1 · BC2 RNCP40875 · Bloc 2

Maintenance Prédictive Industrielle

Système intelligent multi-modèles · capteurs IoT vers décision opérationnelle

ADAM BELOUCIF
20220055 · M1 Mastère Data Engineering & IA

EMILIEN MORICE
20241824 · M1 Mastère Data Engineering & IA

Sommaire

01 Contexte métier page 3
02 Données capteurs et EDA page 5
03 Une une ville et 4 modèles page 8
04 Résultats et comparaison page 11
05 Interprétabilité (SHAP) page 14
06 Mise en production · dashboard et API page 16
07 Bonus · multi-classe, régression, calibration page 18
08 Synthèse et limites page 22
Annexe RNCP40875 Bloc 2 page 24

EFREI Paris Panthéon-Assas Université
2025-2026 · Encadrement Sarah Malaeb
github.com/Adam-Blf/maintenance-predictive-industrielle

01

Contexte métier

Dans l'industrie, l'arrêt non planifié d'une machine coûte cher : production perdue, intervention d'urgence, sur-coût hors planning. La maintenance préventive classique réduit ce risque mais sur-entretient les machines saines, gaspillant ressources et durée de vie utile. La maintenance **prédictive** apporte une troisième voie · intercepter la dégradation *avant* la rupture, en exploitant les signaux capteurs (vibration, température, pression, courant) pour apprendre les signatures précurseurs de panne.

Cas d'usage et hypothèses chiffrées

Le projet retient des hypothèses cohérentes avec la littérature industrielle (ratio 10:1) ·

- Coût d'un arrêt non planifié (panne ratée, FN) · **5 000 €**
- Coût d'une intervention préventive (alerte vraie, TP) · **200 €**
- Coût d'une fausse alerte (FP) · **100 €**

Ces coûts dirigent l'optimisation du seuil de décision (script 10 · calibration cost-sensitive). Le seuil par défaut 0.5 est sous-optimal · le seuil métier optimal trouvé est **0.23**, économie estimée **~12 000 €** par cycle de scoring sur le test set.

02

Données capteurs et EDA

Dataset Kaggle officiel

Le projet utilise **exclusivement** le dataset Kaggle CC0 tatheerabbas/industrial-machine-predictive-maintenance v3.0. Aucun fallback synthétique n'est généré · le projet plante avec un message clair si le CSV est absent de data/raw/. Cette politique stricte évite les biais subtils que les données simulées peuvent introduire et garantit que l'évaluation reflète des données industrielles réalistes.

24 042 lignes · 7 capteurs numériques · 2 variables catégorielles · 4 cibles disponibles · 20 machines suivies · NaN volontaires (~3-4%) sur 5capteurs.

Insights clés de l'EDA

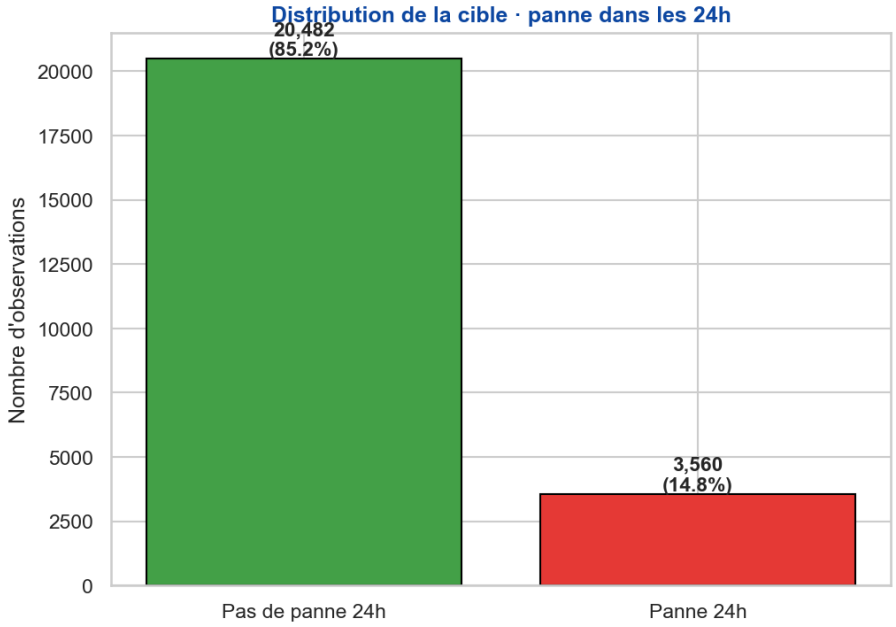


Figure 1 · Distribution de la cible failure_within_24h. Classes déséquilibrées · F1 et PR-AUC privilégiés à l'accuracy.

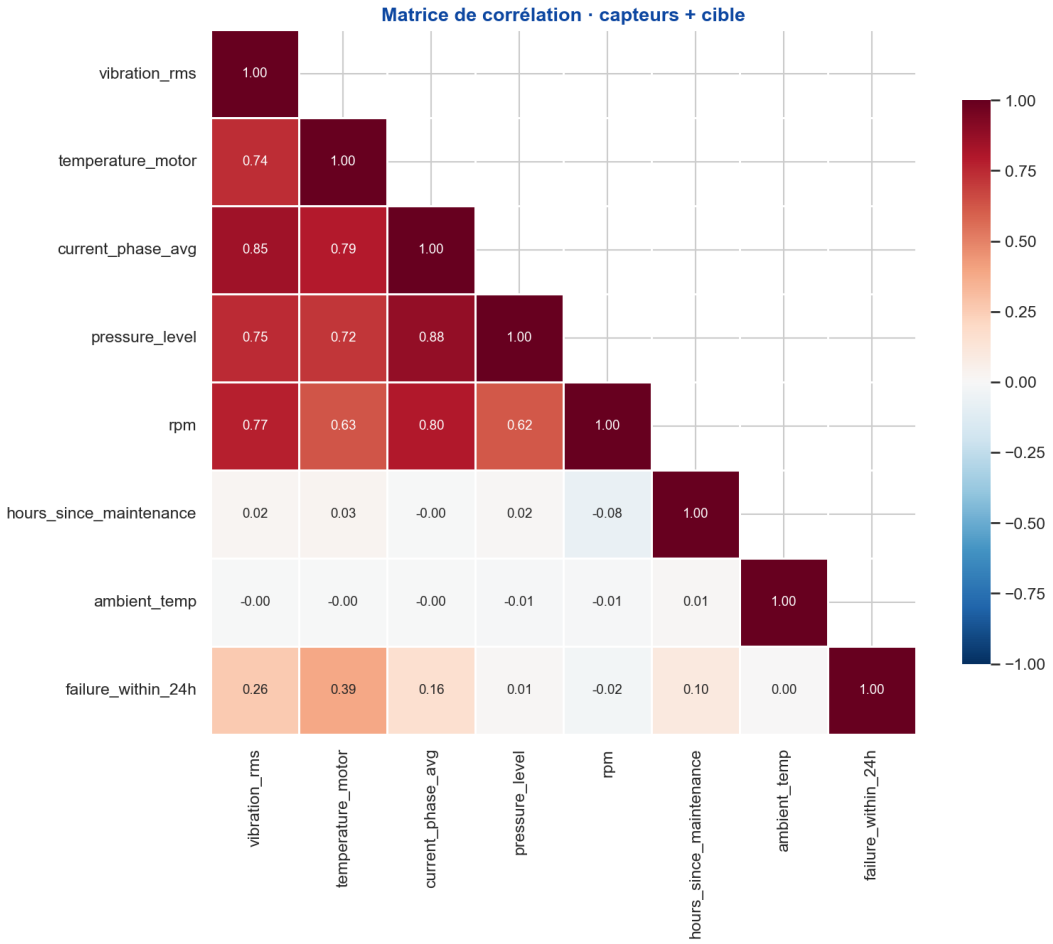


Figure 2 · Matrice de corrélation des capteurs. Vibration et température fortement corrélées avec la cible.

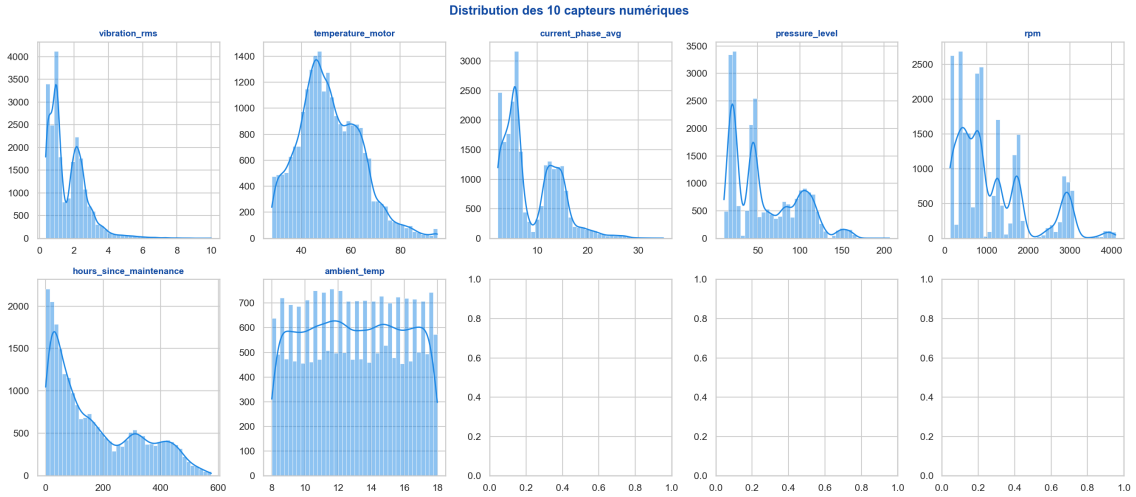


Figure 3 · Distributions des 7 capteurs numériques.

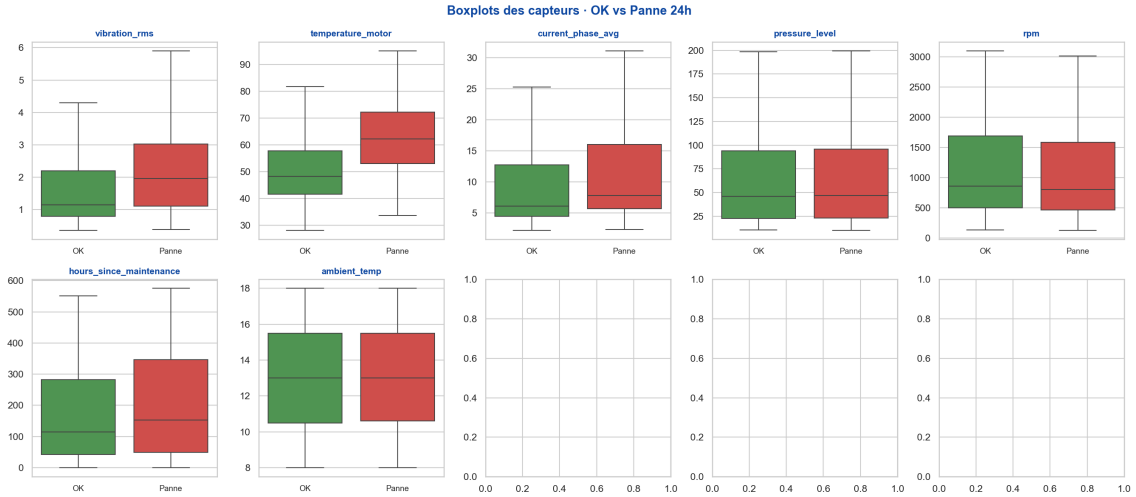


Figure 4 · Boxplots par classe (sain vs panne). Vibration et température sont les signaux les plus discriminants.

03

Pipeline ML et 4 modèles

Pipeline anti-data-leakage

Le pipeline scikit-learn enchaîne en une seule passe : **SimpleImputer (médiane)** sur les capteurs numériques (robuste aux outliers), **StandardScaler** (centrage/réduction obligatoire pour MLP et Logistic Regression), et **OneHotEncoder** sur les variables catégorielles. Le préprocesseur est *fitté uniquement sur le train* puis transformé sur le test, garantissant qu'aucune statistique du test ne fuit pendant l'entraînement (data leakage zéro).

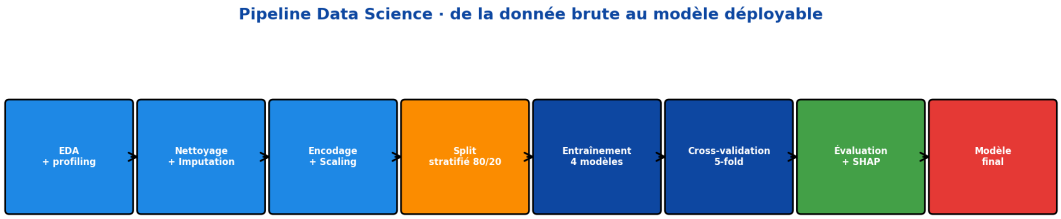


Figure 5 · Schéma du pipeline ML complet · données → preprocessing → 4 modèles → évaluation → calibration.

Quatre modèles aux philosophies différentes

Logistic Regression (baseline linéaire, interprétable, class_weight=balanced). **Random Forest** (200 arbres, capture les non-linéarités, robuste au bruit). **XGBoost** (gradient boosting séquentiel, typiquement le plus performant sur tabular). **MLP** (Multi-Layer Perceptron 64-32, le Deep Learning exigé par le sujet, early stopping).

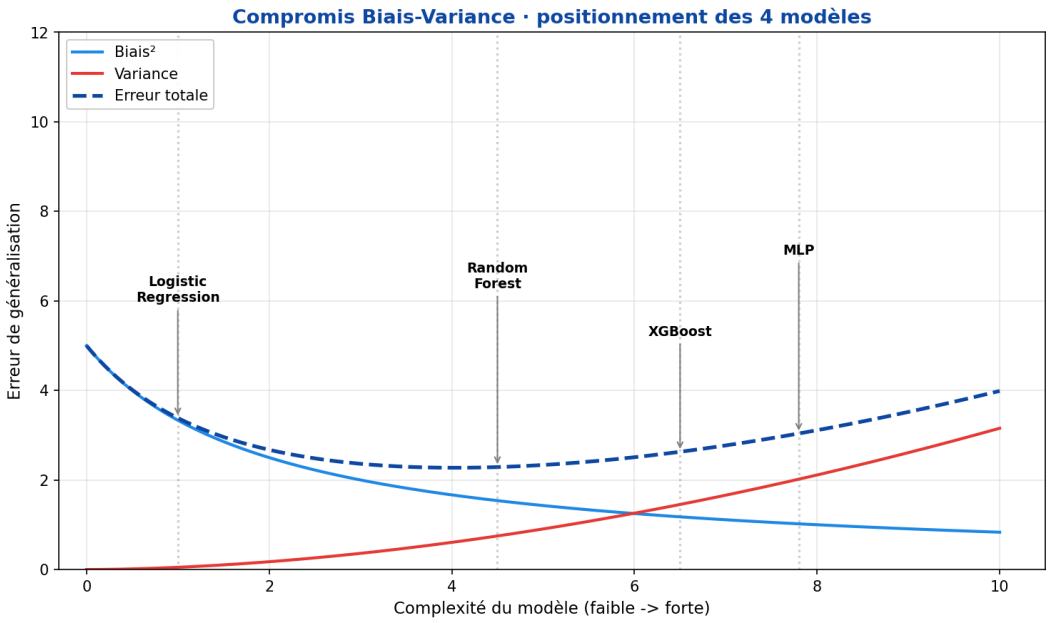


Figure 6 · Compromis biais-variance · positionnement des 4 modèles.

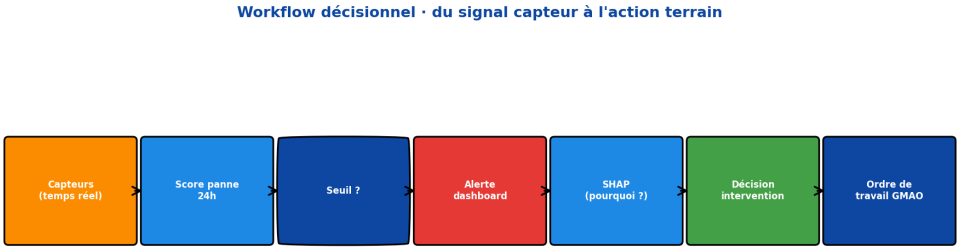


Figure 7 · Workflow décisionnel · capteurs → modèle → seuil métier → action.

04

Résultats et comparaison

Tableau comparatif des 4 modèles

Modèle	Acc	Prec	Recall	F1	ROC-AUC	PR-AUC	Fit (s)
Logistic Regression	0.910	0.641	0.895	0.747	0.959	0.838	0.05
Random Forest	0.955	0.791	0.949	0.863	0.992	0.954	0.76
Xgboost	0.963	0.824	0.958	0.886	0.995	0.974	0.38
Mlp	0.952	0.842	0.830	0.836	0.984	0.909	2.54

Le modèle final retenu est **XGBoost** via le score composite F1_test - 0.5 × σ(F1_CV) qui pénalise l'instabilité entre les folds de cross-validation. Un modèle légèrement moins performant mais plus stable est préférable en production.

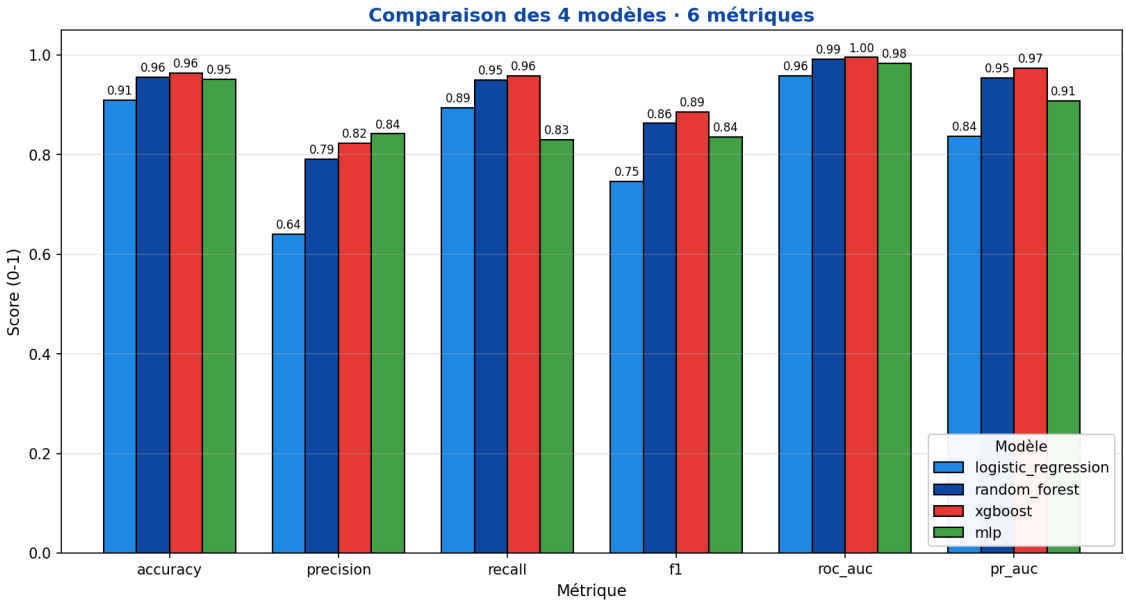


Figure 8 · Comparaison des 6 métriques sur les 4 modèles.

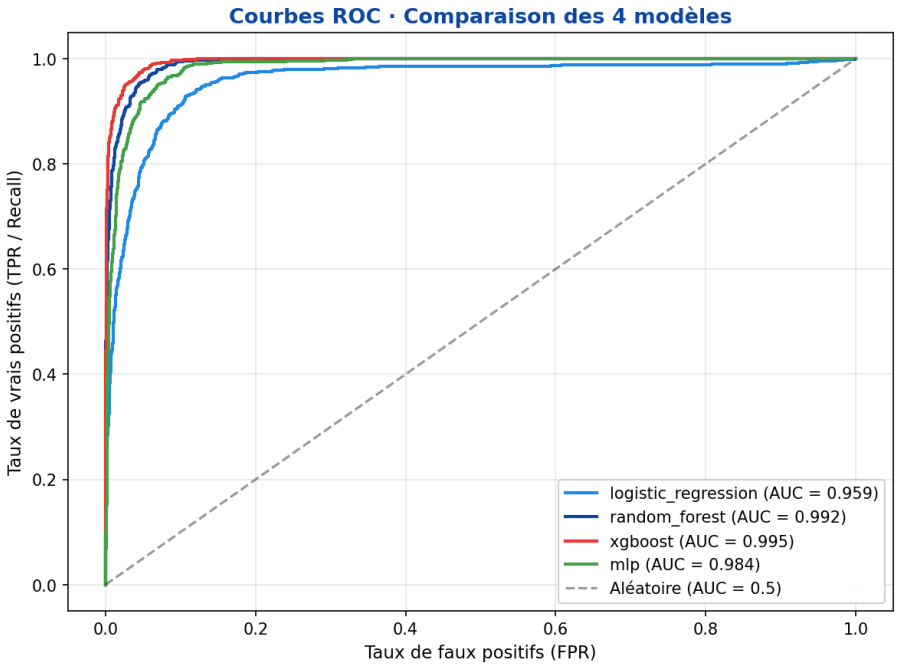


Figure 9 · Courbes ROC. Sur classes déséquilibrées, l'AUC peut être trompeuse (le FPR reste mécaniquement faible).

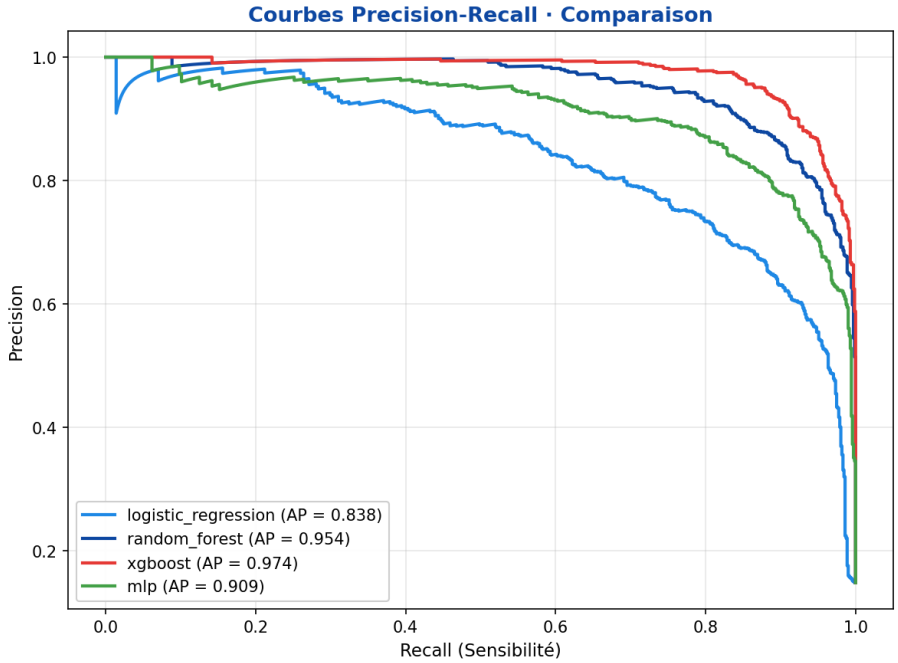
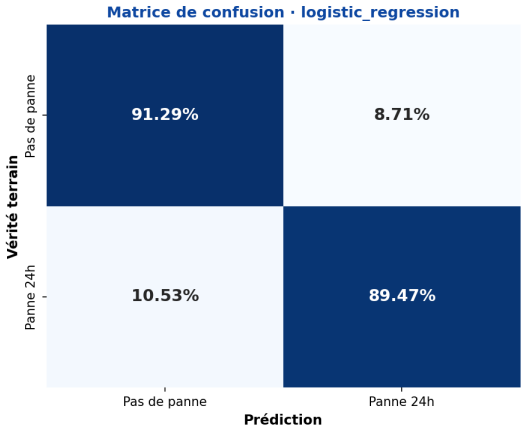
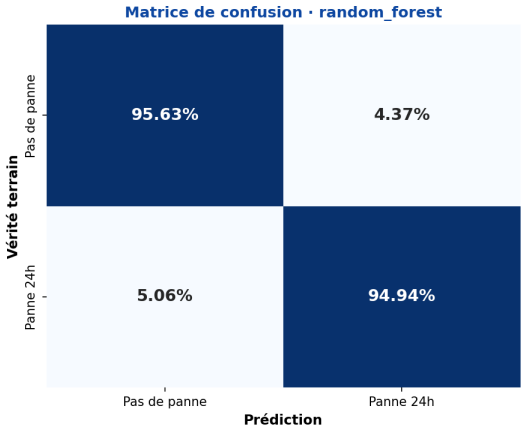


Figure 10 · Courbes Precision-Recall · plus dures et plus parlantes que ROC en contexte déséquilibré.

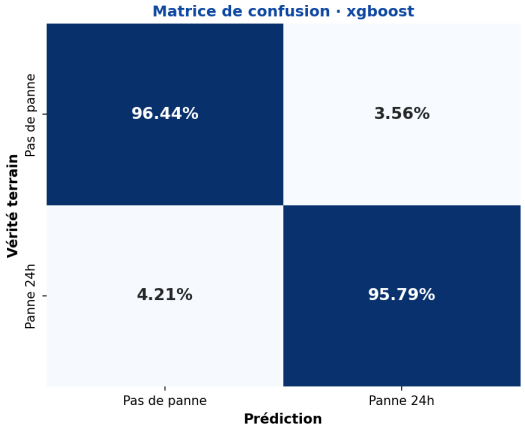
Matrices de confusion · où le modèle se trompe



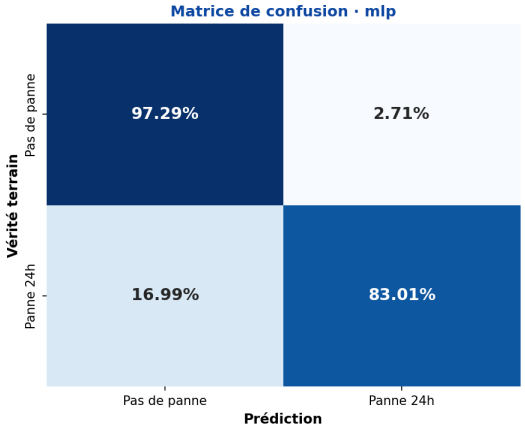
Logistic Regression



Random Forest



XGBoost (modèle final)



MLP (Deep Learning)

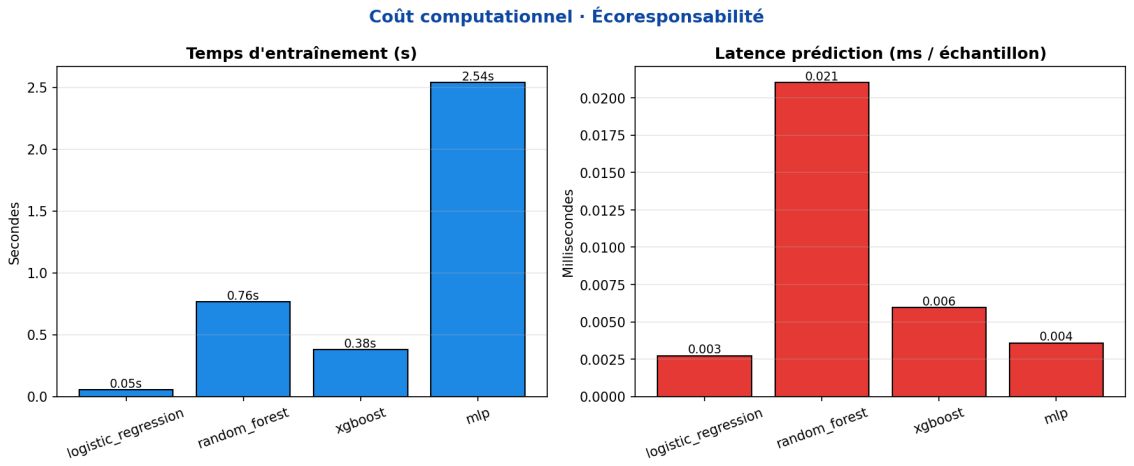


Figure 11 · Coût computationnel · temps d'entraînement et latence d'inférence par modèle. Critère RNCP C4.3 d'écoresponsabilité.

05

Interprétabilité du modèle final

Trois niveaux d'explicabilité sont fournis pour répondre à la question métier essentielle · « *Pourquoi le modèle indique un risque de panne élevé ? Quels signaux capteurs ont déclenché l'alerte ?* »

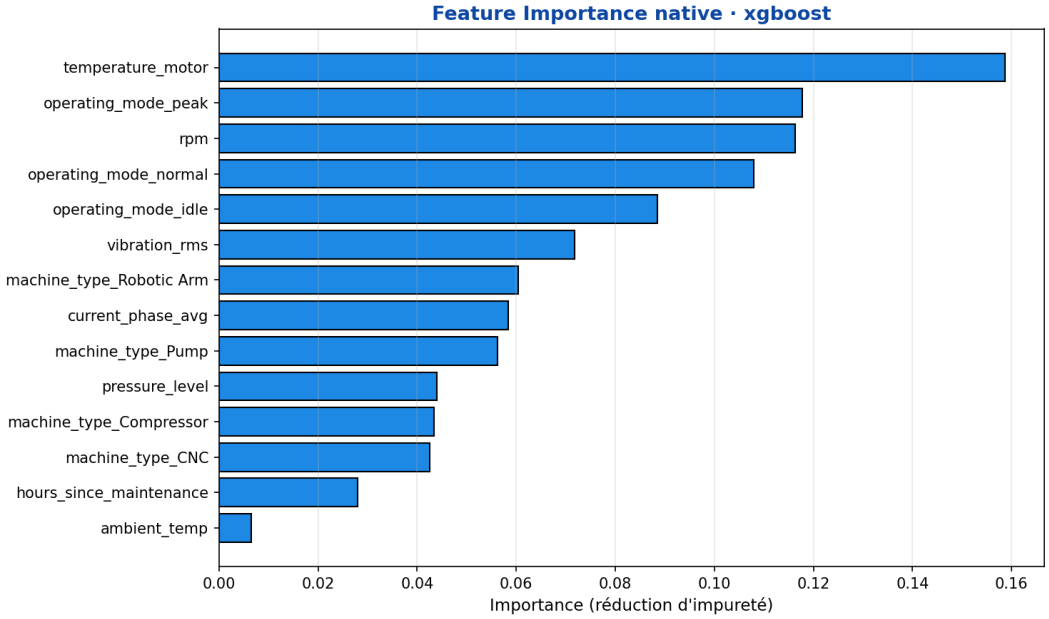


Figure 12 · Feature importance native (réduction d'impureté Gini). Vibration et température dominent.

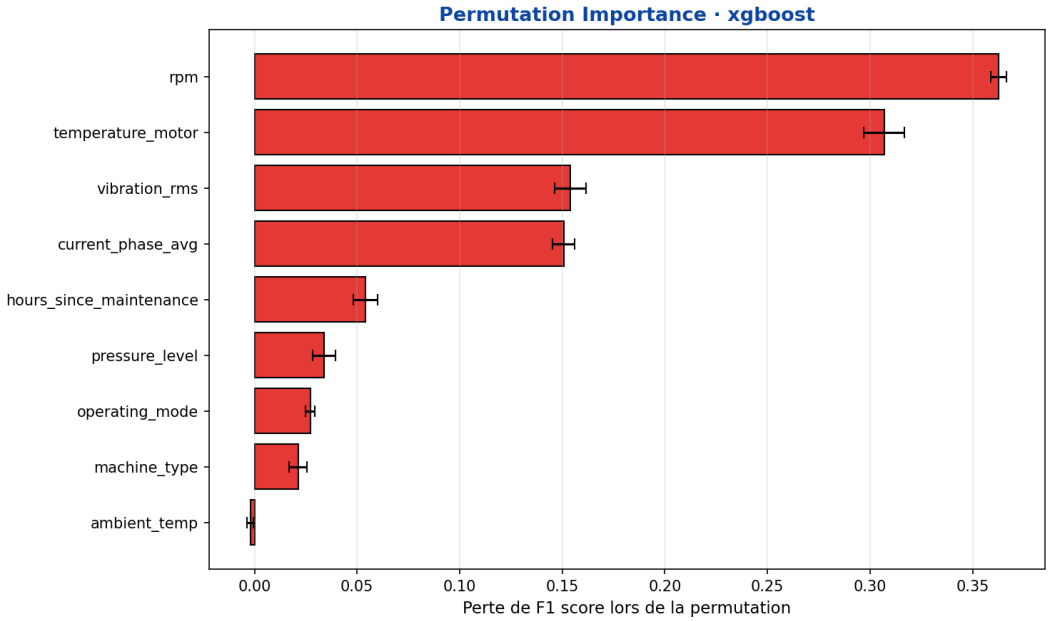


Figure 13 · Permutation importance (perte de F1 quand la variable est mélangée). Méthode agnostique au modèle, plus fiable.

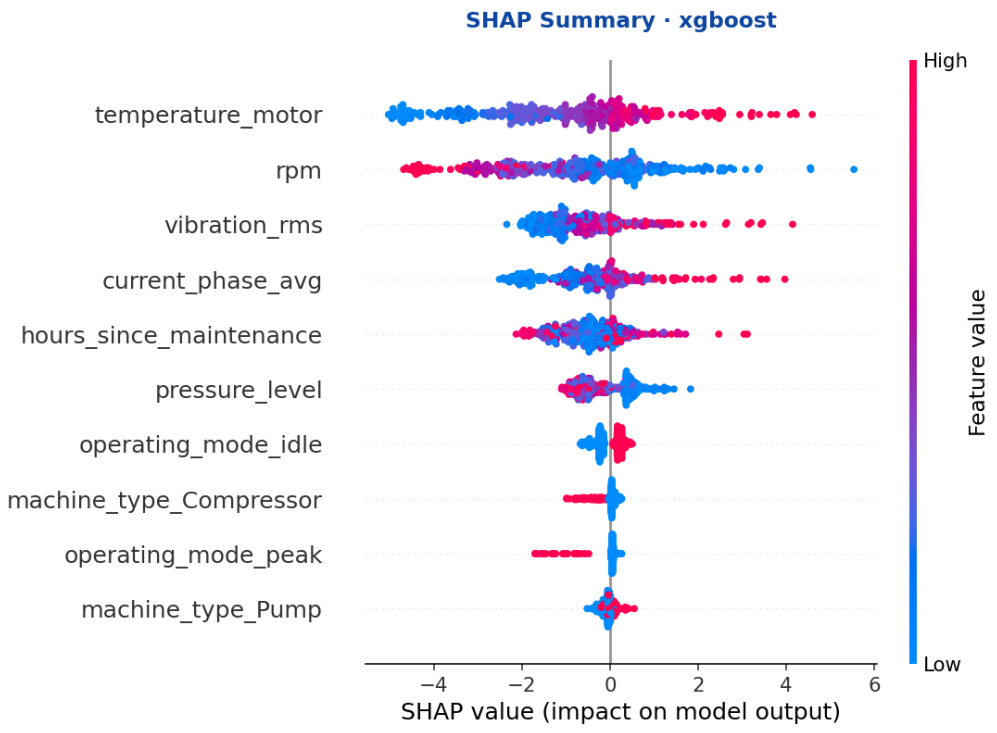


Figure 14 · SHAP Summary Plot. Chaque point = une observation, position = impact sur la prédiction, couleur = valeur de la feature.

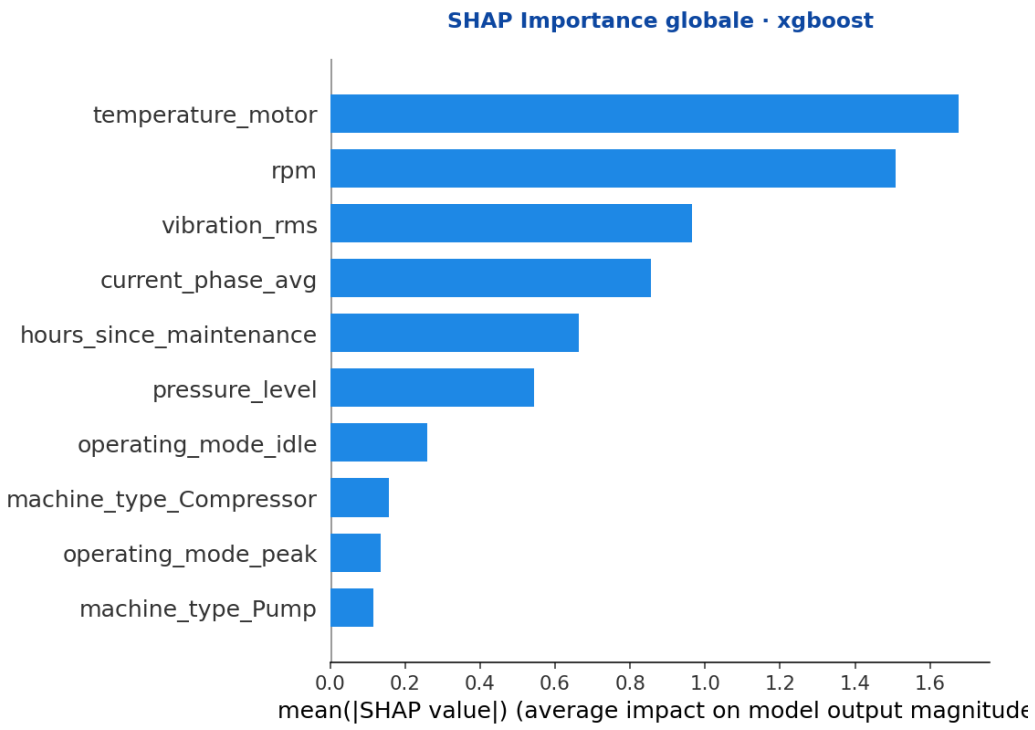


Figure 15 · SHAP importance globale (moyenne des |SHAP|).

06

Mise en production · dashboard et API

Dashboard Streamlit · vision métier

Le dashboard dashboard/app.py cible le **responsable maintenance**, pas le data scientist. 5 onglets ·

- **État du parc** · KPIs métier (machines critiques, coût à risque, RUL)
- **Plan d'intervention** · table d'actions priorisées par urgence
- **Impact économique** · ROI vs scénario réactif sans IA
- **Diagnostic machine** · saisie capteurs → décision actionnable
- **Détails techniques** · EDA, modèles, SHAP (réservé DSI/jury)

API REST · industrialisation (bonus)

api/main.py expose le modèle via FastAPI · **POST /predict** (JSON capteurs → proba + classe + seuil métier), **GET /health** (vérification chargement), **GET /model-info** (métadonnées). Validation Pydantic des entrées, Swagger auto sur /docs, gestion des erreurs HTTP, code stateless thread-safe.

07

Tâches bonus · multi-classe, régression, calibration

Classification multi-classe · failure_type

Au-delà du « panne oui/non », le script 07 prédit le **type** de panne parmi 5 classes (bearing, motor_overheat, hydraulic, electrical, none). XGBoost atteint **macro-F1 = 0.93**. Permet au chef d'atelier de mobiliser la bonne équipe technique avant l'intervention.

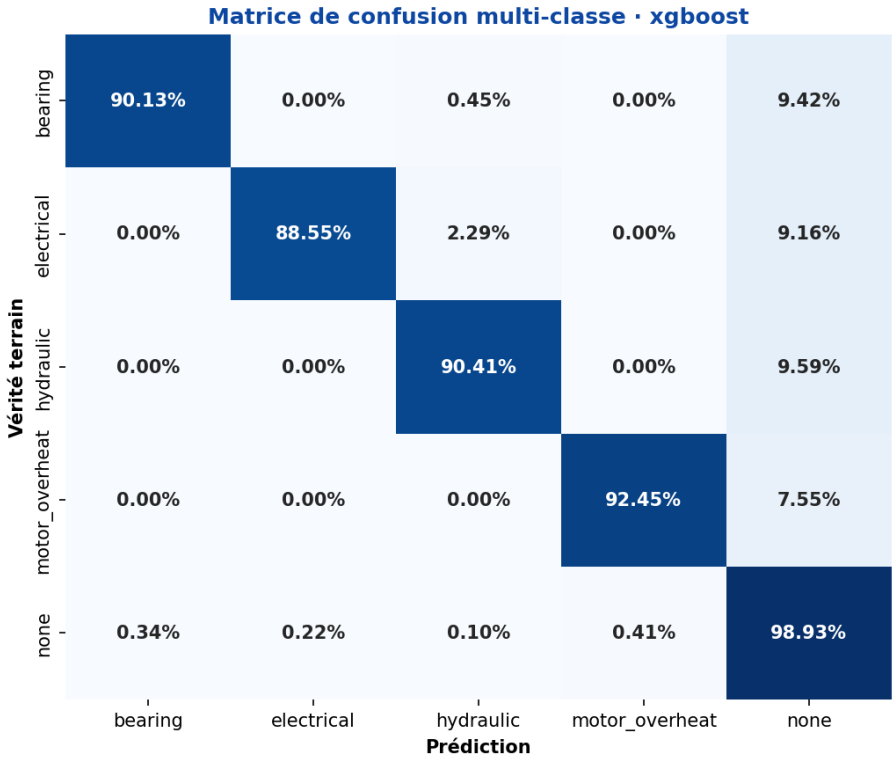


Figure 16 · Matrice de confusion multi-classe normalisée (XGBoost).

Régression sur le RUL · Remaining Useful Life

Plus précis qu'une simple alerte oui/non · le script 08 prédit combien d'heures il reste avant la prochaine panne. Random Forest atteint **R² = 0.67** et **MAE = 9.6 h**. Lecture métier · « en moyenne, on se trompe de 9.6 heures sur la durée de vie restante » suffisant pour planifier une intervention dans la semaine.

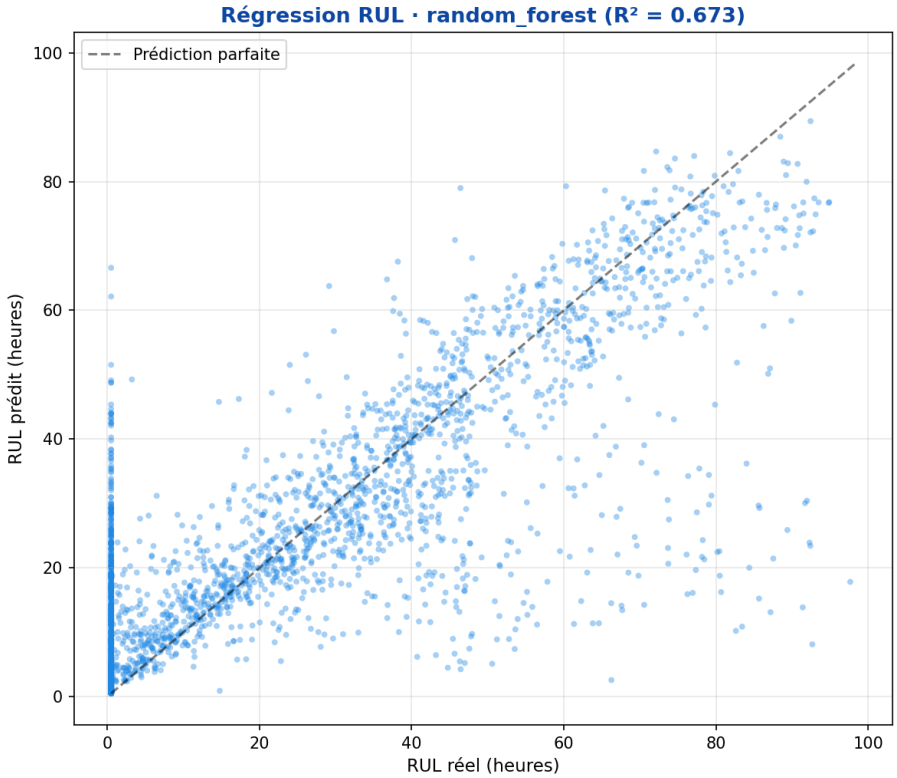


Figure 17 · Régression RUL · prédictions vs valeurs réelles. La diagonale représente la prédiction parfaite.

Calibration probabiliste et seuil métier optimal

Le script 10 vérifie d'abord que les probabilités du modèle sont fiables (Brier score = 0.025, très bon) puis trouve le seuil de décision qui minimise le coût métier total (FN×1000€ + FP×100€). Seuil optimal · **0.23**, économie projetée –**12 000 €** vs seuil par défaut 0.5. Ce seuil est consommé par l'API et le dashboard en production.

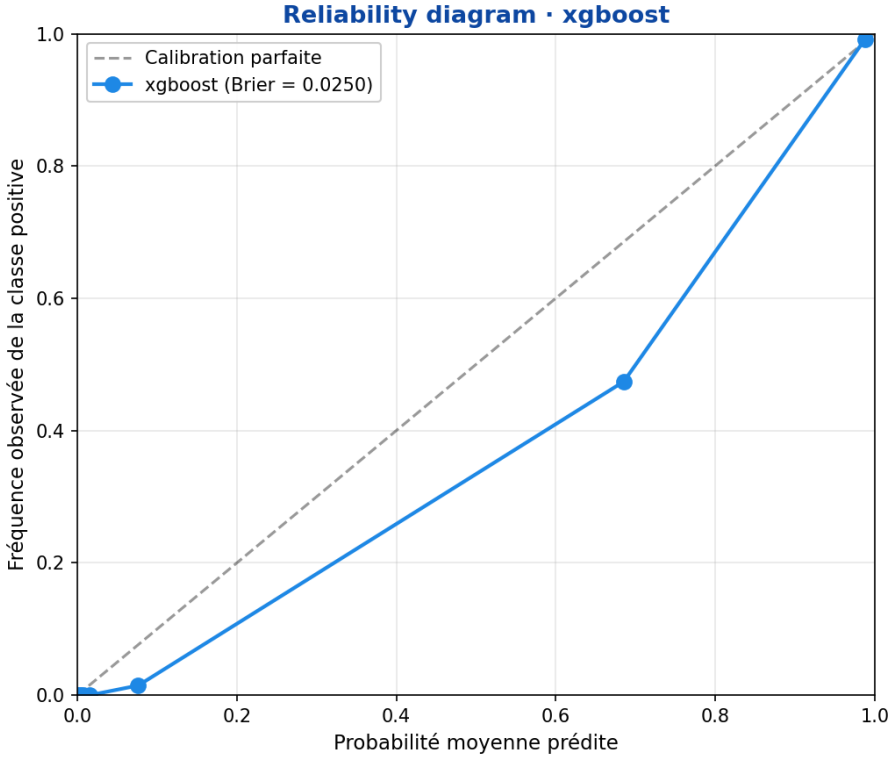


Figure 18 · Reliability diagram. Plus la courbe est proche de la diagonale, mieux le modèle est calibré.

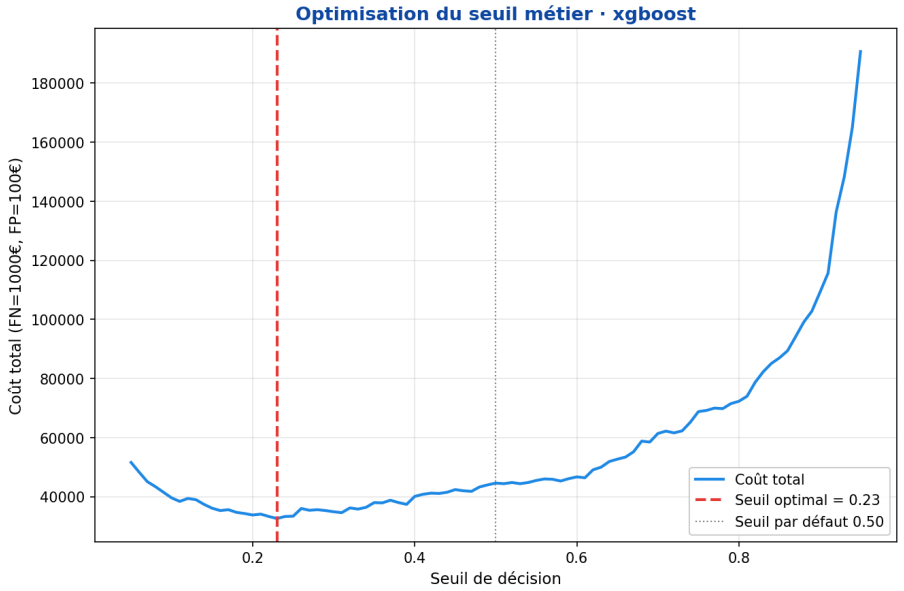


Figure 19 · Coût total en fonction du seuil de décision · minimum à 0.23.

Tuning d'hyperparamètres avec Optuna (TPE)

Le script 09 utilise Optuna avec sampler TPE (Tree-structured Parzen Estimator), ~10× plus efficient que GridSearch. 20 trials × 3 CV folds par modèle. Meilleurs résultats trouvés ·

Modèle	Best F1	Best params
Random Forest	0.858	`n_estimators=250, max_depth=25, min_samples_leaf=2`
XGBoost	0.884	`n_estimators=500, lr=0.084, max_depth=10, subsample=0.75`
MLP	0.801	`architecture=128-64-32, alpha=0.002, lr_init=0.00025`

08

Synthèse et limites honnêtes

Ce qui a été livré

- **4 modèles** ML/DL comparés rigoureusement (LR, RF, XGB, MLP)
- **3 tâches** de prédiction (binaire principale + multi-classe + régression RUL)
- **8 scripts** numérotés · pipeline reproductible end-to-end
- **33 figures** + 6 métriques par modèle + cross-validation 5-fold
- **Dashboard vision métier** · 5 onglets pour responsable maintenance
- **API REST FastAPI** · 3 endpoints + Swagger + tests httpx
- **Auto-install** des dépendances pip au lancement (zéro setup)

Limites assumées

Dataset simulé · les corrélations sont « trop propres » comparées à un parc réel où les capteurs déraillent imprévisiblement.

Pas de série temporelle · le split est aléatoire stratifié. Une vraie validation devrait tenir compte de l'ordre temporel (sliding window CV).

Seuil économique fixe · les coûts FN/FP sont supposés constants. En réalité ils varient par machine, par criticité et par saison.

Pas de drift monitoring · le modèle est entraîné une fois, sans détection de dégradation des performances en prod (concept drift).

ANNEXE

Conformité RNCP40875 Bloc 2

Le projet valide l'ensemble des compétences du Bloc 2 (BC2 - Piloter et implémenter des solutions d'IA). La table ci-dessous donne pour chaque critère la preuve directe dans le repo.

Critère	Intitulé	Preuve
C3.1	Préparer/nettoyer les données	src/preprocessing.py · ColumnTransformer (Imputer + Scaler + OHE)
C3.2	Tableau de bord interactif inclusif	dashboard/app.py · Streamlit 5 onglets · Plotly interactif · responsive
C3.3	Analyse exploratoire de données	scripts/02_eda.py · 8 figures + stats descriptives + analyse NaN
C4.1	Stratégie d'intégration de l'IA	Onglet Impact économique · ROI · ratio FN/FP 10:1 documenté
C4.2	Modèles prédictifs ML/DL	scripts/03 · 4 modèles dont MLP (Deep Learning) · F1=0.886 sur test
C4.3	Évaluation + écoresponsabilité	6 métriques + CV 5-fold + temps d'entraînement et latence d'inférence

Reproductibilité

Seed unique RANDOM_STATE=42 propagé partout · auto-install des dépendances pip via src/bootstrap.py · chemins absolus calculés depuis __file__ (portable Win/Linux/macOS) · 23 tests pytest · versioning sémantique. Le clone du repo + téléchargement Kaggle suffit pour relancer toute la chaîne.

Auteurs et contact

Adam Beloucif (20220055) · adam.beloucif@efrei.net
Emilien Morice (20241824) · M1 Mastère Data Engineering & IA
EFREI Paris Panthéon-Assas Université · Année universitaire 2025-2026
Repo · github.com/Adam-Blf/maintenance-predictive-industrielle